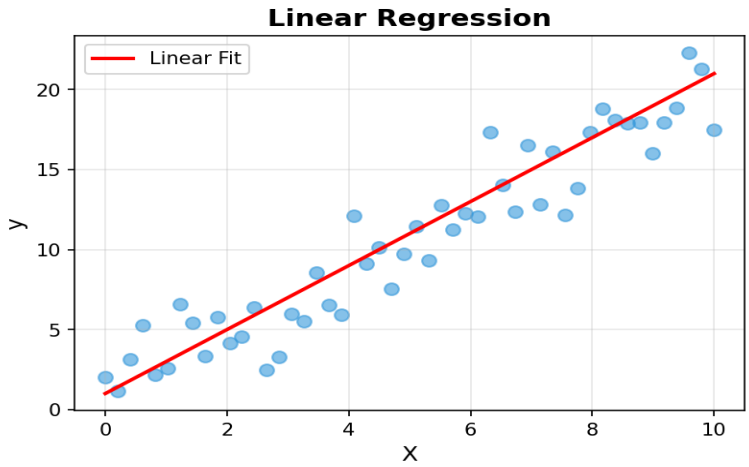
MACHINE LEARNING COMPLETE CHEAT SHEET

TABLE OF CONTENTS

1. SUPERVISED LEARNING ALGORITHMS

1.1 Linear Regression

Predicts continuous values by fitting a linear relationship between features and target.



Formula: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \varepsilon$

Use Cases: Price prediction, sales forecasting, trend analysis

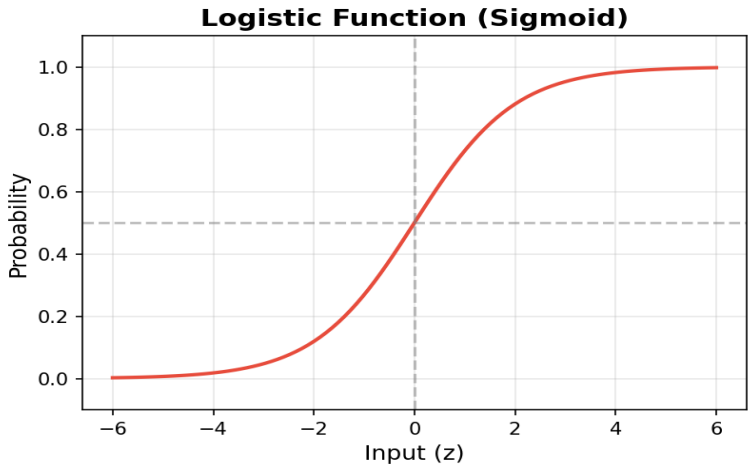
Pros: Simple, interpretable, fast | **Cons:** Assumes linearity, sensitive to outliers

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

1.2 Logistic Regression

Binary classification using sigmoid function to predict probabilities.

Mehtab Ansari



Formula: $P(y=1) = 1 / (1 + e^{-z})$ where $z = \beta_0 + \beta_1 x_1 + ...$

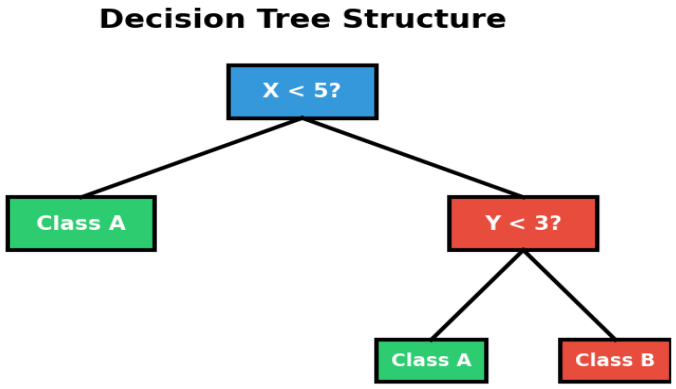
Use Cases: Email spam detection, disease diagnosis, customer churn

Pros: Probabilistic output, works well for linearly separable data

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict_proba(X_test)
```

1.3 Decision Trees

Tree-like model making decisions based on feature thresholds.



Key Metrics: Gini Impurity, Entropy, Information Gain

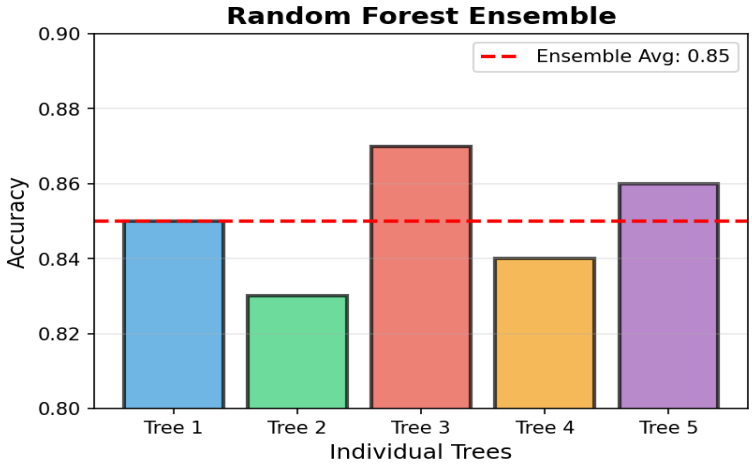
Use Cases: Customer segmentation, risk assessment, medical diagnosis

Pros: Interpretable, handles non-linear data | **Cons:** Prone to overfitting

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)
```


1.4 Random Forest

Ensemble of decision trees using bagging to reduce variance.



Key Idea: Multiple trees vote for final prediction (classification) or average (regression)

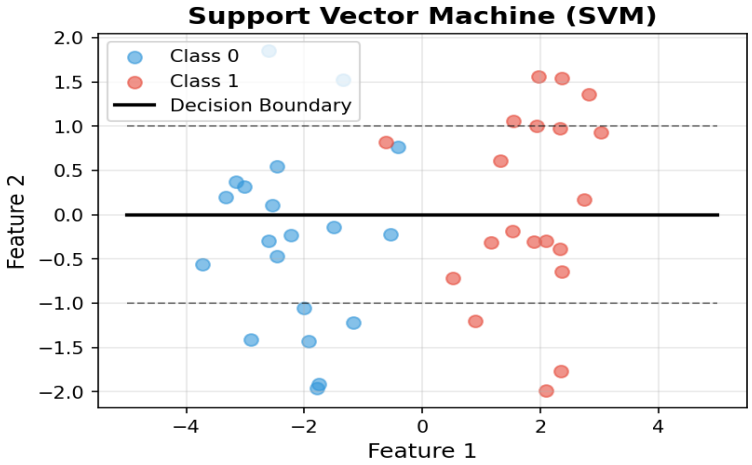
Use Cases: Feature importance, fraud detection, recommendation systems

Pros: Reduces overfitting, robust | **Cons:** Less interpretable, computationally expensive

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
importances = model.feature_importances_
```

1.5 Support Vector Machine (SVM)

Finds optimal hyperplane that maximizes margin between classes.



Kernels: Linear, RBF (Radial Basis Function), Polynomial, Sigmoid

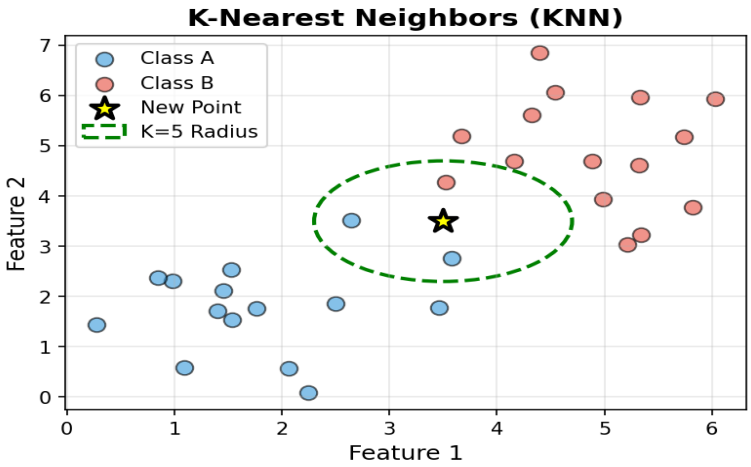
Use Cases: Image classification, text categorization, handwriting recognition

Pros: Effective in high dimensions | **Cons:** Slow on large datasets

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
```

1.6 K-Nearest Neighbors (KNN)

Classifies based on majority vote of K nearest neighbors.



Distance Metrics: Euclidean, Manhattan, Minkowski, Hamming

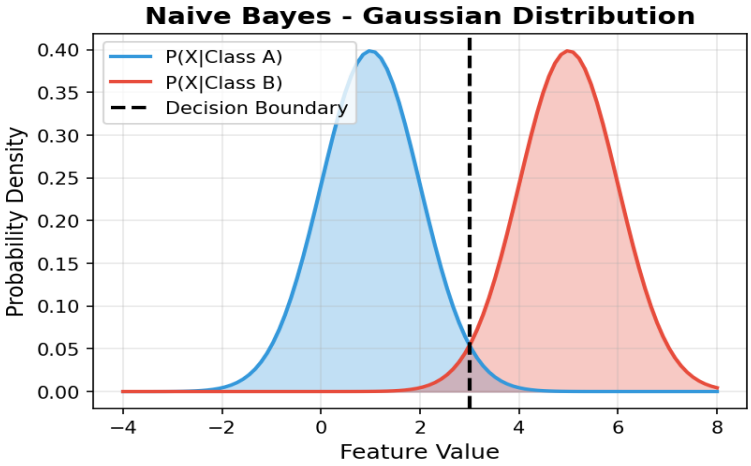
Use Cases: Pattern recognition, anomaly detection, recommender systems

Pros: Simple, no training phase | **Cons:** Slow prediction, sensitive to scale

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

1.7 Naive Bayes

Probabilistic classifier based on Bayes' theorem with independence assumption.



Formula: $P(y|X) = P(X|y) \times P(y) / P(X)$

Types: Gaussian NB, Multinomial NB, Bernoulli NB

Use Cases: Text classification, spam filtering, sentiment analysis

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
```

1.8 Gradient Boosting (XGBoost, LightGBM)

Sequential ensemble method building trees to correct previous errors.

Popular Libraries: XGBoost, LightGBM, CatBoost

Use Cases: Kaggle competitions, ranking problems, complex predictions

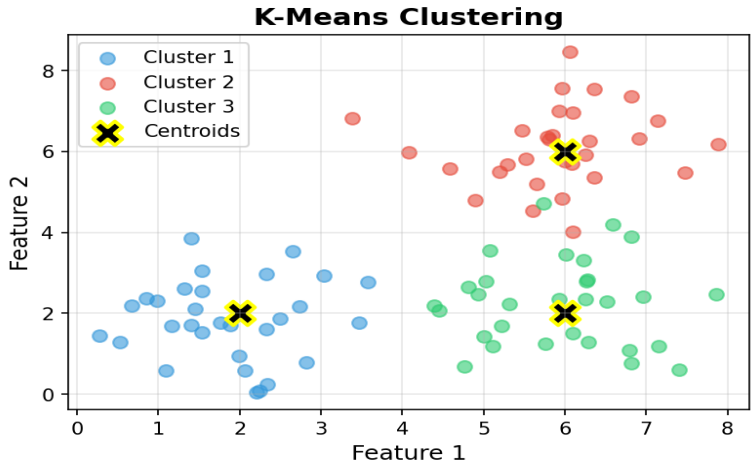
Pros: High accuracy, handles missing data | **Cons:** Prone to overfitting, requires tuning

```
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=100, learning_rate=0.1)
model.fit(X_train, y_train)
```


2. UNSUPERVISED LEARNING ALGORITHMS

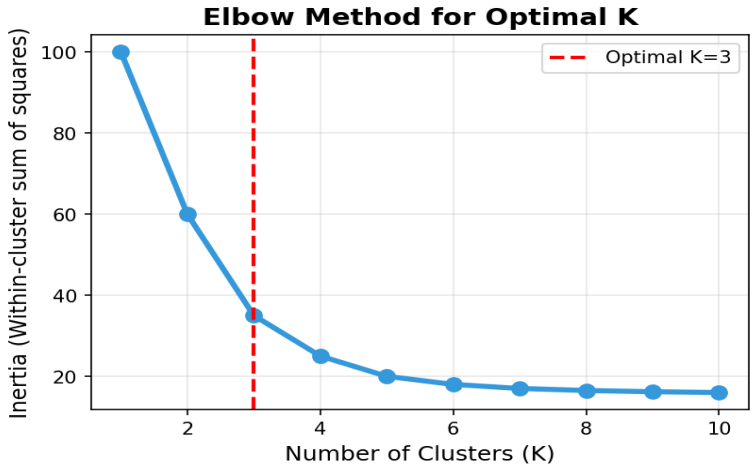
2.1 K-Means Clustering

Partitions data into K clusters by minimizing within-cluster variance.



Algorithm: 1) Initialize centroids, 2) Assign points, 3) Update centroids, 4) Repeat

Choosing K: Elbow method, Silhouette score



Use Cases: Customer segmentation, image compression, anomaly detection

```
from sklearn.cluster import KMeans
model = KMeans(n_clusters=3, random_state=42)
clusters = model.fit_predict(X)
```

2.2 Hierarchical Clustering

Builds tree of clusters (dendrogram) using agglomerative or divisive approach.

Linkage Methods: Single, Complete, Average, Ward

Use Cases: Gene sequencing, document clustering, taxonomy

```
from sklearn.cluster import AgglomerativeClustering
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
clusters = model.fit_predict(X)
```

2.3 DBSCAN (Density-Based Clustering)

Finds clusters of arbitrary shape based on density of points.

Parameters: eps (neighborhood radius), min_samples (minimum points)

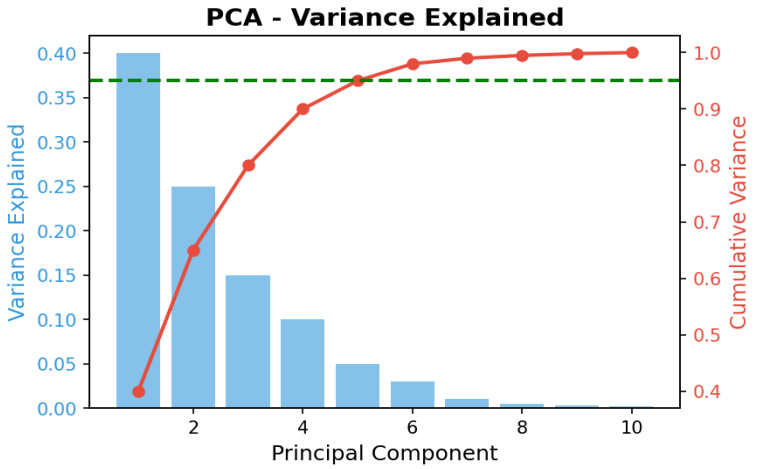
Use Cases: Spatial data, outlier detection, arbitrary shapes

Pros: Finds outliers, no need to specify K | **Cons:** Sensitive to parameters

```
from sklearn.cluster import DBSCAN
model = DBSCAN(eps=0.5, min_samples=5)
clusters = model.fit_predict(X)
```

2.4 Principal Component Analysis (PCA)

Dimensionality reduction by projecting data onto principal components.



Goal: Maximize variance while reducing dimensions

Use Cases: Visualization, noise reduction, feature extraction

Variance Explained: Choose components that explain 95%+ variance

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)
```

2.5 t-SNE (t-Distributed Stochastic Neighbor Embedding)

Non-linear dimensionality reduction for visualization in 2D/3D.

Use Cases: Visualizing high-dimensional data, exploring clusters

Note: Primarily for visualization, not feature extraction

```
from sklearn.manifold import TSNE
tsne = TSNE(n_components=2, perplexity=30)
X_embedded = tsne.fit_transform(X)
```

2.6 Association Rule Learning (Apriori, FP-Growth)

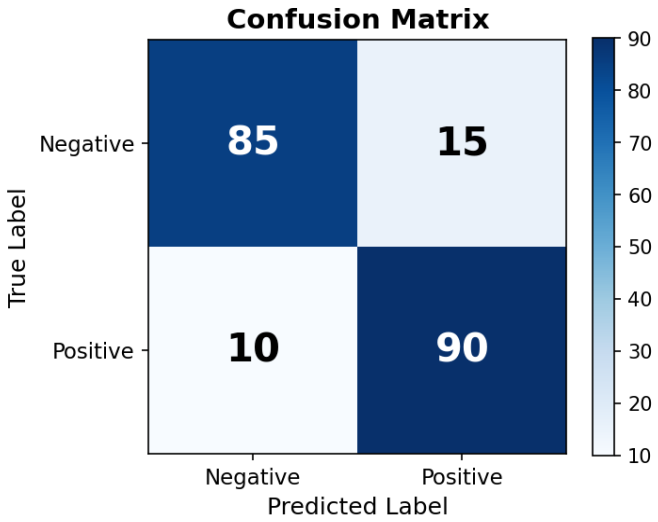
Discovers relationships between items in transactional data.

Metrics: Support, Confidence, Lift

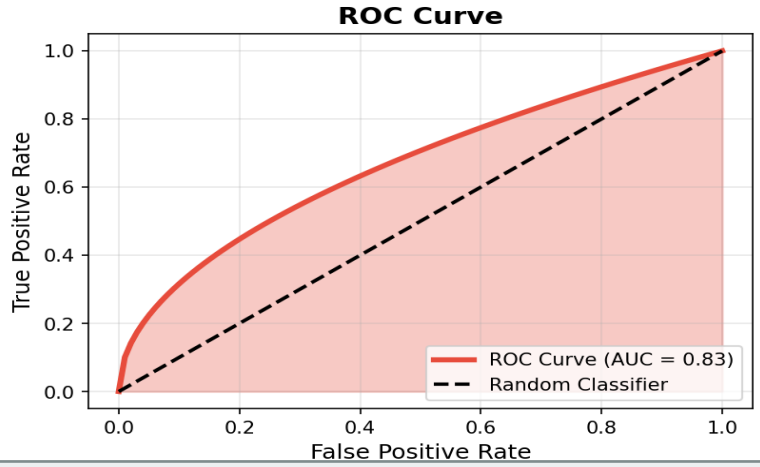
Use Cases: Market basket analysis, recommendation systems

3. MODEL EVALUATION METRICS

3.1 Classification Metrics



Metric	Formula	When to Use
Accuracy	(TP+TN) / (TP+TN+FP+FN)	Balanced datasets
Precision	TP / (TP+FP)	Minimize false positives
Recall (Sensitivity)	TP / (TP+FN)	Minimize false negatives
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	Balance precision & recall
ROC-AUC	Area under ROC curve	Compare models




```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_sc
ore
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
```

3.2 Regression Metrics

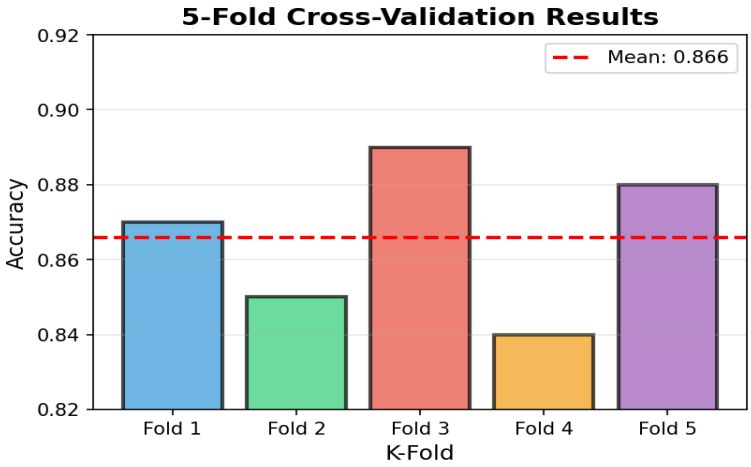
Metric	Formula	Interpretation
MAE	Σ|y - ■| / n	Average absolute error

MSE	Σ(y - ■)² / n	Penalizes large errors
RMSE	√(MSE)	Same units as target
R² Score	1 - (SS_res / SS_tot)	Variance explained (0-1)
Adjusted R²	Adjusted for features	Better for multiple features

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
mae = mean_absolute_error(y_true, y_pred)
rmse = mean_squared_error(y_true, y_pred, squared=False)
r2 = r2_score(y_true, y_pred)
```

3.3 Cross-Validation

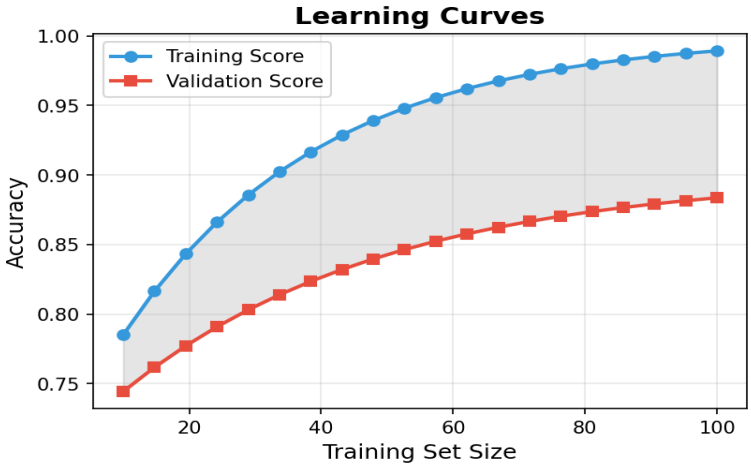
Evaluate model performance on different subsets of data.

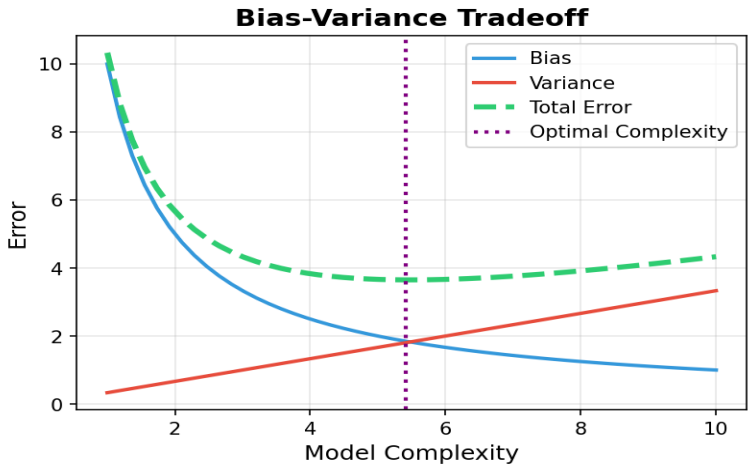


Types: K-Fold, Stratified K-Fold, Leave-One-Out, Time Series Split

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f"CV Accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})")
```

3.4 Learning Curves & Bias-Variance




Bias-Variance Tradeoff
Bias
Variance
Total Error
Optimal Complexity
Error
Model Complexity
0
2
4
6
8
10
2
4
6
8
10

4. FEATURE ENGINEERING

4.1 Feature Scaling

Standardization (Z-score): $(x - \mu) / \sigma \rightarrow$ Mean=0, Std=1

Normalization (Min-Max): $(x - min) / (max - min) \rightarrow$ Range [0,1]

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
# Standardization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Normalization
normalizer = MinMaxScaler()
X_normalized = normalizer.fit_transform(X)
```

4.2 Encoding Categorical Variables

Label Encoding: Convert categories to integers (0, 1, 2, ...)

One-Hot Encoding: Create binary columns for each category

Target Encoding: Replace with mean of target for each category

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
# Label Encoding
le = LabelEncoder()
y_encoded = le.fit_transform(y)
# One-Hot Encoding
from pandas import get_dummies
X_encoded = get_dummies(X, columns=['category_col'])
```

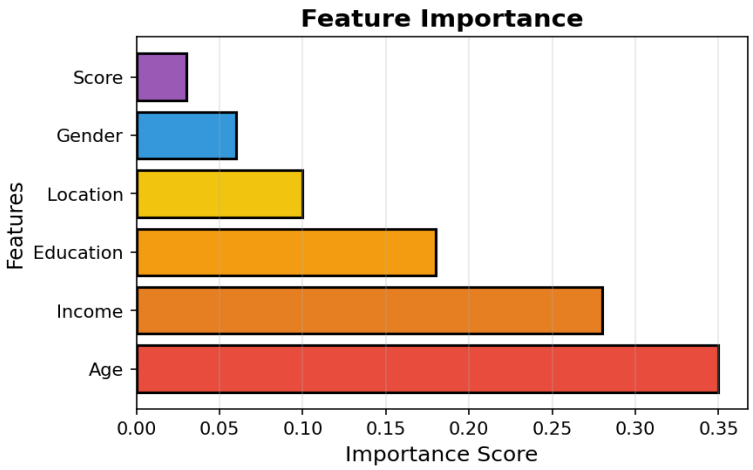
4.3 Handling Missing Data

Strategies: Drop rows/columns, Mean/Median/Mode imputation, KNN imputation, Forward/Backward fill

```
from sklearn.impute import SimpleImputer
# Mean imputation
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
# Or drop missing
X_clean = X.dropna()
```

4.4 Feature Selection

Methods: Correlation matrix, Recursive Feature Elimination (RFE), Feature importance, Mutual information



```
from sklearn.feature_selection import SelectKBest, f_classif, RFE
# Select K best features
selector = SelectKBest(f_classif, k=10)
X_selected = selector.fit_transform(X, y)
# RFE
rfe = RFE(estimator=model, n_features_to_select=10)
X_rfe = rfe.fit_transform(X, y)
```

4.5 Feature Creation

Polynomial Features: x■, x■ → x■, x■, x■², x■x■, x■²

Binning: Convert continuous to categorical (age → age_group)

Log Transform: Handle skewed distributions

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
```


5. DATA PREPROCESSING PIPELINE

5.1 Train-Test Split

Separate data to avoid overfitting and evaluate generalization.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

5.2 Handling Imbalanced Data

Techniques: SMOTE (oversampling), undersampling, class weights, ensemble methods

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_balanced, y_balanced = smote.fit_resample(X_train, y_train)
```

5.3 Pipeline Creation

Streamline preprocessing and modeling steps to prevent data leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(n_estimators=100))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

5.4 Hyperparameter Tuning

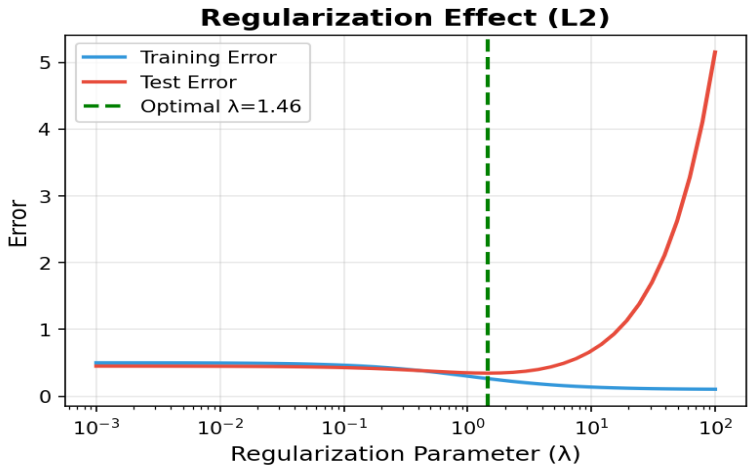
Grid Search: Exhaustive search over parameter grid

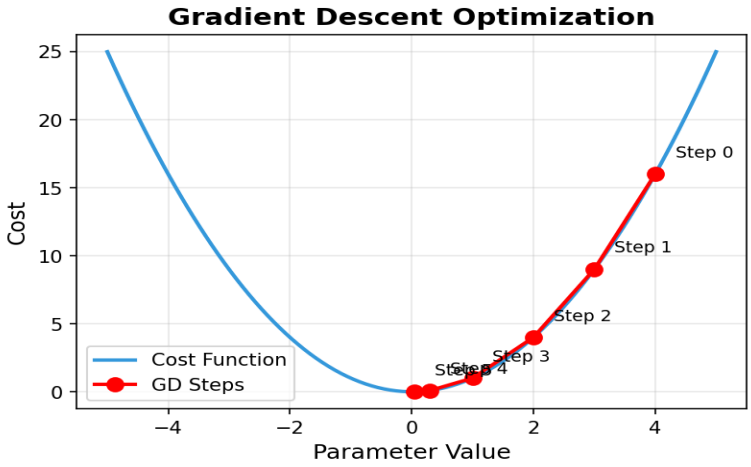
Random Search: Random sampling of parameters (faster)

Bayesian Optimization: Intelligent search using probability

```
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5, 10]
}
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
best_model = grid_search.best_estimator_
```

5.5 Regularization & Optimization





6. KEY FORMULAS & CONCEPTS

Concept	Formula / Description
Bias-Variance Tradeoff	Total Error = $Bias^2$ + Variance + Irreducible Error
Learning Rate	Controls step size in gradient descent (typically 0.001-0.1)
Regularization (L1)	Loss + $\lambda \Sigma\|w\|$ (Lasso - feature selection)
Regularization (L2)	Loss + $\lambda \Sigma w^2$ (Ridge - prevents large weights)
Entropy	$H(X) = -\Sigma p(x) \log_2(p(x))$
Gini Impurity	$Gini = 1 - \Sigma p_i^2$
Information Gain	IG = H(parent) - Σ(weight × H(child))
Gradient Descent	$\theta = \theta - \alpha \nabla J(\theta)$
Euclidean Distance	$d = \sqrt{(\Sigma(x_i - y_i)^2)}$
Manhattan Distance	$d = \Sigma\|x_i - y_i\|$
Cosine Similarity	$\cos(\theta) = (A \cdot B) / (\|\|A\|\| \|\|B\|\|)$

6.1 Important Concepts

• **Overfitting:** Model learns noise in training data → Poor generalization

• **Underfitting:** Model too simple → High bias, can't capture patterns

• **Curse of Dimensionality:** As features increase, data becomes sparse

• **No Free Lunch:** No single algorithm works best for all problems

• **Feature Importance:** Which features contribute most to predictions

• **Model Interpretability:** Understanding why model makes predictions

• **Ensemble Learning:** Combine multiple models for better performance

Created by: Mehtab Ansari

Machine Learning Complete Reference Guide

Visit scikit-learn.org for detailed documentation